Project:

GDP_Predictor

COURSE TITLE:

Artificial Intelligence

PROFESSOR:

Dr. Naveed Anwar Butt

SUBMITTED BY:

Ayesha Noor 21017119-053

DATE OF SUBMISSION:

Wednesday, March 8TH 2023

Loading Dataset:

GDP_Predictor

In [31]:
```
#Importing Libraries
import pandas as pd
import numpy as np
```

In [32]:
```
#Loading the Dataset
data = pd.read_csv("D:\sem_3\AI\countries of the world.csv")
```

Data Description:

- ***Head Function:***

In [70]:
```
#shows the Dataset from the start
data.head(5)
```

Out[70]:

	Country	Region	Population	Area	Density	Coastline_Ratio	Net_Migration	Infant_Mortality	GDP_per_Capita	Literacy	Phones	Arable	Crops	O
0	Afghanistan	ASIA (EX. NEAR EAST)	31056997	647500	48.0	0.00	23.06	163.07	700.0	36.0	3.2	12.13	0.22	8
1	Albania	EASTERN EUROPE	3581655	28748	124.6	1.26	-4.93	21.52	4500.0	86.5	71.2	21.09	4.42	7
2	Algeria	NORTHERN AFRICA	32930091	2381740	13.8	0.04	-0.39	31.00	6000.0	70.0	78.1	3.22	0.25	9
3	American Samoa	OCEANIA	57794	199	290.4	58.29	-20.71	9.27	8000.0	97.0	259.5	10.00	15.00	7
4	Andorra	WESTERN EUROPE	71201	468	152.1	0.00	6.60	4.05	19000.0	100.0	497.2	2.22	0.00	9

- ***Tail Function:***

In [71]:
```
#shows the Dataset at the end
data.tail(4)
```

Out[71]:

	Country	Region	Population	Area	Density	Coastline_Ratio	Net_Migration	Infant_Mortality	GDP_per_Capita	Literacy	Phones	Arable	Crops	O
223	Western Sahara	NORTHERN AFRICA	273008	266000	1.0	0.42	0.0	0.00	2500.0	67.24	100.2	0.02	0.00	9
224	Yemen	NEAR EAST	21456188	527970	40.6	0.36	0.0	61.50	800.0	50.20	37.2	2.78	0.24	9
225	Zambia	SUB-SAHARAN AFRICA	11502010	752614	15.3	0.00	0.0	88.29	800.0	80.60	8.2	7.08	0.03	9
226	Zimbabwe	SUB-SAHARAN AFRICA	12236805	390580	31.3	0.00	0.0	67.69	1900.0	90.70	26.8	8.32	0.34	9

- ***Info Function:***

Before Fixing….

```
In [6]: #shows the infromation of Dataset (No. of Col, Column names, Datatypes)
        data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 227 entries, 0 to 226
Data columns (total 20 columns):
 #   Column                              Non-Null Count  Dtype
---  ------                              --------------  -----
 0   Country                             227 non-null    object
 1   Region                              227 non-null    object
 2   Population                          227 non-null    int64
 3   Area (sq. mi.)                      227 non-null    int64
 4   Pop. Density (per sq. mi.)          227 non-null    object
 5   Coastline (coast/area ratio)        227 non-null    object
 6   Net migration                       224 non-null    object
 7   Infant mortality (per 1000 births)  224 non-null    object
 8   GDP ($ per capita)                  226 non-null    float64
 9   Literacy (%)                        209 non-null    object
 10  Phones (per 1000)                   223 non-null    object
 11  Arable (%)                          225 non-null    object
 12  Crops (%)                           225 non-null    object
 13  Other (%)                           225 non-null    object
 14  Climate                             205 non-null    object
 15  Birthrate                           224 non-null    object
 16  Deathrate                           223 non-null    object
 17  Agriculture                         212 non-null    object
 18  Industry                            211 non-null    object
 19  Service                             212 non-null    object
dtypes: float64(1), int64(2), object(17)
memory usage: 35.6+ KB
```

Fixing Column Names:

```
In [7]: #<<<<<<<<<<<<<<<<<<<<Fixing Column Names>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
        #Many columns in the dataset have long names, we will change them to be shorter and better describtive.
        data.columns = (["Country", "Region", "Population", "Area", "Density", "Coastline_Ratio", "Net_Migration", "Infant_Mortality",
                         "GDP_per_Capita", "Literacy", "Phones", "Arable", "Crops", "Other", "Climate", "Birth_Rate", "Death_Rate",
                         "Agriculture", "Industry", "Service"])
```

Fixing Datatypes:

In [8]:
```
#<<<<<<<<<<<<<<<<<<<<<<<Fixing Datatypes>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
#Many columns in that dataset have _object_ as type. We will fix this by assigning float/string types to them.
data.Country = data.Country.astype('category')

data.Region = data.Region.astype('category')

data.Density = data.Density.astype(str)
data.Density = data.Density.str.replace(",",".").astype(float)

data.Coastline_Ratio = data.Coastline_Ratio.astype(str)
data.Coastline_Ratio = data.Coastline_Ratio.str.replace(",",".").astype(float)

data.Net_Migration = data.Net_Migration.astype(str)
data.Net_Migration = data.Net_Migration.str.replace(",",".").astype(float)

data.Infant_Mortality = data.Infant_Mortality.astype(str)
data.Infant_Mortality = data.Infant_Mortality.str.replace(",",".").astype(float)

data.Literacy = data.Literacy.astype(str)
data.Literacy = data.Literacy.str.replace(",",".").astype(float)

data.Phones = data.Phones.astype(str)
data.Phones = data.Phones.str.replace(",",".").astype(float)

data.Arable = data.Arable.astype(str)
data.Arable = data.Arable.str.replace(",",".").astype(float)

data.Crops = data.Crops.astype(str)
data.Crops = data.Crops.str.replace(",",".").astype(float)

data.Other = data.Other.astype(str)
data.Other = data.Other.str.replace(",",".").astype(float)
```


```
data.Other = data.Other.astype(str)
data.Other = data.Other.str.replace(",",".").astype(float)
```


```
data.Climate = data.Climate.astype(str)
data.Climate = data.Climate.str.replace(",",".").astype(float)

data.Birth_Rate = data.Birth_Rate.astype(str)
data.Birth_Rate = data.Birth_Rate.str.replace(",",".").astype(float)

data.Death_Rate = data.Death_Rate.astype(str)
data.Death_Rate = data.Death_Rate.str.replace(",",".").astype(float)

data.Agriculture = data.Agriculture.astype(str)
data.Agriculture = data.Agriculture.str.replace(",",".").astype(float)

data.Industry = data.Industry.astype(str)
data.Industry = data.Industry.str.replace(",",".").astype(float)

data.Service = data.Service.astype(str)
data.Service = data.Service.str.replace(",",".").astype(float)
```

- ***Info Function:***

After Fixing Column names and Datatypes…

```
In [9]: ##shows the infromation of Dataset (No. of Col, Column names, Datatypes) after changing column names and datatypes
        data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 227 entries, 0 to 226
Data columns (total 20 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Country           227 non-null    category
 1   Region            227 non-null    category
 2   Population        227 non-null    int64
 3   Area              227 non-null    int64
 4   Density           227 non-null    float64
 5   Coastline_Ratio   227 non-null    float64
 6   Net_Migration     224 non-null    float64
 7   Infant_Mortality  224 non-null    float64
 8   GDP_per_Capita    226 non-null    float64
 9   Literacy          209 non-null    float64
 10  Phones            223 non-null    float64
 11  Arable            225 non-null    float64
 12  Crops             225 non-null    float64
 13  Other             225 non-null    float64
 14  Climate           205 non-null    float64
 15  Birth_Rate        224 non-null    float64
 16  Death_Rate        223 non-null    float64
 17  Agriculture       212 non-null    float64
 18  Industry          211 non-null    float64
 19  Service           212 non-null    float64
dtypes: category(2), float64(16), int64(2)
memory usage: 43.0 KB
```

Eliminating Missing Values:

- ***Isnull.sum Function:***

Before eliminating missing values…

```
n [11]: #Dispaly All the null Values in the Dataset
        print(data.isnull().sum())
```

```
Country              0
Region               0
Population           0
Area                 0
Density              0
Coastline_Ratio      0
Net_Migration        3
Infant_Mortality     3
GDP_per_Capita       1
Literacy            18
Phones               4
Arable               2
Crops                2
Other                2
Climate             22
Birth_Rate           3
Death_Rate           4
Agriculture         15
Industry            16
Service             15
dtype: int64
```

- ***Fillna Pandas Function:***

```
In [12]: #Filling the missing Datapoints in the dataset
         data['Net_Migration'].fillna(0, inplace=True)
         data['Infant_Mortality'].fillna(0, inplace=True)
         data['GDP_per_Capita'].fillna(2500, inplace=True)
         data['Literacy'].fillna(data.groupby('Region')['Literacy'].transform('mean'), inplace= True)
         data['Phones'].fillna(data.groupby('Region')['Phones'].transform('mean'), inplace= True)
         data['Arable'].fillna(0, inplace=True)
         data['Crops'].fillna(0, inplace=True)
         data['Other'].fillna(0, inplace=True)
         data['Climate'].fillna(0, inplace=True)
         data['Birth_Rate'].fillna(data.groupby('Region')['Birth_Rate'].transform('mean'), inplace= True)
         data['Death_Rate'].fillna(data.groupby('Region')['Death_Rate'].transform('mean'), inplace= True)
         data['Agriculture'].fillna(0.17, inplace=True)
         data['Service'].fillna(0.8, inplace=True)
         data['Industry'].fillna((1 - data['Agriculture'] - data['Service']), inplace= True)
```

- ***Isnull.sum Function:***

After filling the missing values in the dataset…

```
In [13]: #Dispaly All the null Values in the Dataset after filling the missing values
         print(data.isnull().sum())

Country             0
Region              0
Population          0
Area                0
Density             0
Coastline_Ratio     0
Net_Migration       0
Infant_Mortality    0
GDP_per_Capita      0
Literacy            0
Phones              0
Arable              0
Crops               0
Other               0
Climate             0
Birth_Rate          0
Death_Rate          0
Agriculture         0
Industry            0
Service             0
dtype: int64
```

Exploratory Data Analysis (EDA):

- ***Describe Function:***

```
In [15]: #Describe function is used to get a descriptive statistics summary (mean, count, S.D, percentiles, and min-max) values.
         data.describe()
```

Out[15]:

	Population	Area	Density	Coastline_Ratio	Net_Migration	Infant_Mortality	GDP_per_Capita	Literacy	Phones	Arable	C
count	2.270000e+02	2.270000e+02	227.000000	227.000000	227.000000	227.000000	227.000000	227.000000	227.000000	227.000000	227.00
mean	2.874028e+07	5.982270e+05	379.047137	21.165330	0.037621	35.037709	9658.149780	83.387497	235.215128	13.675551	4.52
std	1.178913e+08	1.790282e+06	1660.185825	72.286863	4.856712	35.388330	10038.230555	19.251553	226.204958	13.046722	8.33
min	7.026000e+03	2.000000e+00	0.000000	0.000000	-20.990000	0.000000	500.000000	17.600000	0.200000	0.000000	0.00
25%	4.376240e+05	4.647500e+03	29.150000	0.100000	-0.905000	7.825000	1900.000000	75.300000	38.500000	3.095000	0.19
50%	4.786994e+06	8.660000e+04	78.800000	0.730000	0.000000	20.910000	5500.000000	92.500000	176.200000	10.380000	1.01
75%	1.749777e+07	4.418110e+05	190.150000	10.345000	0.980000	55.335000	15700.000000	98.000000	382.900000	20.000000	4.43
max	1.313974e+09	1.707520e+07	16271.500000	870.660000	23.060000	191.190000	55100.000000	100.000000	1035.600000	62.110000	50.68

- ***Show Function(Dtale):***

```
In [12]: dtale.show(data)
```

```
D:\anaconda3\lib\site-packages\dtale\views.py:755: FutureWarning:

['Net migration', 'Infant mortality (per 1000 births)', 'Literacy (%)', 'Phones (per 1000)', 'Arable (%)', 'Crops (%)', 'Other (%)', 'Climate', 'Birthrate', 'Deathrate', 'Agriculture', 'Industry', 'Service'] did not aggregate successfully. If any error is raised this will raise in a future version of pandas. Drop these columns/ops to avoid this warning.
```

D-TALE Actions Visualize Highlight Settings

20 / 227

	Country	Region	Population	Area (sq. mi.)	Pop. Density (per sq. mi.)	Coastline (coast/area ratio)
0	Afghanistan	ASIA (EX. NEAR EAST)	31056997	647500	48,0	0,(
1	Albania	EASTERN EUROPE	3581655	28748	124,6	1,2
2	Algeria	NORTHERN AFRICA	32930091	2381740	13,8	0,(
3	American Samoa	OCEANIA	57794	199	290,4	58,2
4	Andorra	WESTERN EUROPE	71201	468	152,1	0,(
5	Angola	SUB-SAHARAN AFRICA	12127071	1246700	9,7	0,
6	Anguilla	LATIN AMER. & CARIB	13477	102	132,1	59,(
7	Antigua & Barbuda	LATIN AMER. & CARIB	69108	443	156,0	34,5
8	Argentina	LATIN AMER. & CARIB	39921833	2766890	14,4	0,
9	Armenia	C.W. OF IND. STATES	2976372	29800	99,9	0,(
10	Aruba	LATIN AMER. & CARIB	71891	193	372,5	35,4
11	Australia	OCEANIA	20264082	7686850	2,6	0,3
12	Austria	WESTERN EUROPE	8192880	83870	97,7	0,(
13	Azerbaijan	C.W. OF IND. STATES	7961619	86600	91,9	0,(

- ***Profile Report Function (PandasProfiling):***

```
In [18]: profile = ProfileReport(data, title = "Pandas Profiling Report")
         profile
```

Summarize dataset: 100% 353/353 [01:20<00:00, 3.70it/s, Completed]

Generate report structure: 100% 1/1 [00:10<00:00, 10.71s/it]

Render HTML: 100% 1/1 [00:19<00:00, 19.60s/it]

Pandas Profiling Report — Overview Variables Interactions Correlations Missing values Sample

Overview

Overview Alerts 25 Reproduction

Dataset statistics		Variable types	
Number of variables	20	Categorical	2

Overview

Overview | Alerts 25 | Reproduction

Dataset statistics	
Number of variables	20
Number of observations	227
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	43.0 KiB
Average record size in memory	193.8 B

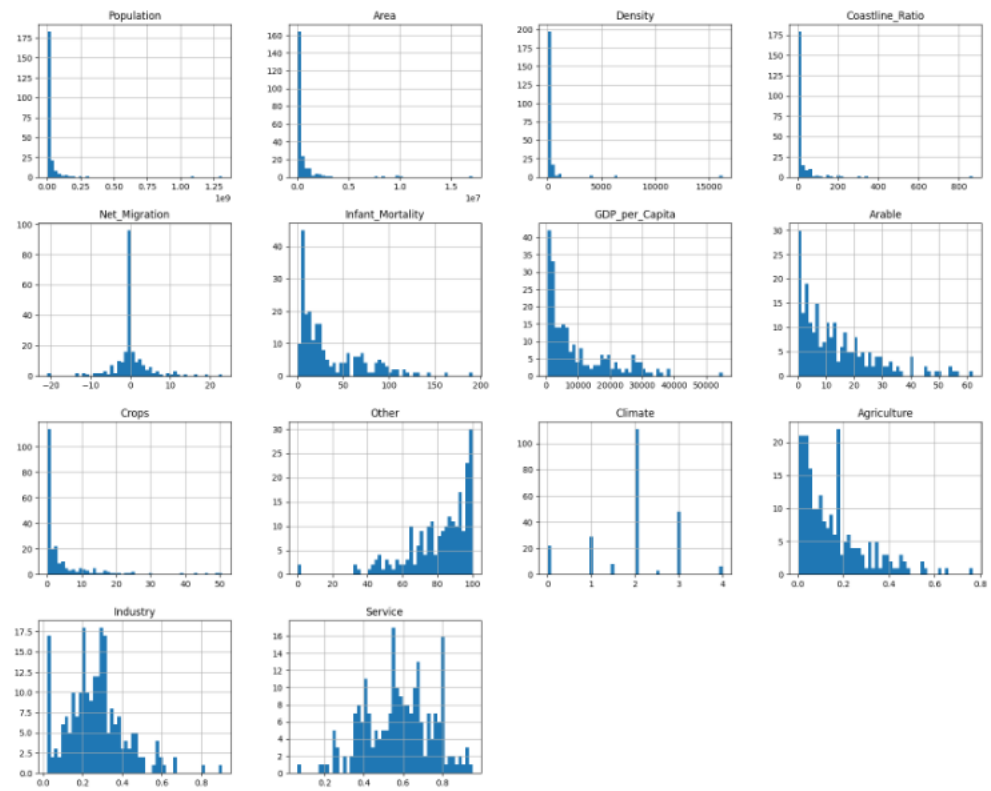
Variable types	
Categorical	2
Numeric	18

Variables

Histograms:

Histogram

[13]:
```
import matplotlib.pyplot as plt
%matplotlib inline

data.hist(bins=50, figsize=(20, 16))
plt.show()
```



Coorelations:

Coorelation

In [48]:
```
correlation_data = data.corr()
correlation_data['GDP_per_Capita'].sort_values(ascending=True)
```

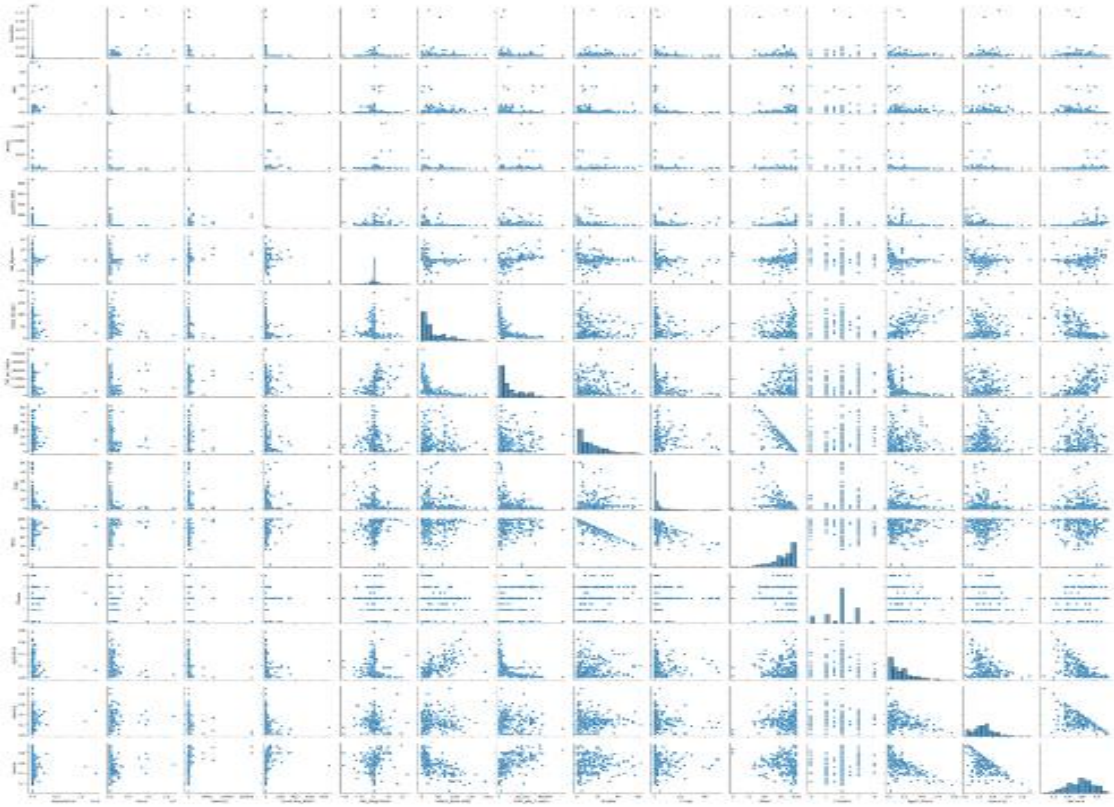
Out[48]:
```
Birth_Rate         -0.642626
Infant_Mortality   -0.587497
Agriculture        -0.568344
Crops              -0.216853
Death_Rate         -0.194767
Industry           -0.064812
Population         -0.038510
Arable              0.021341
Coastline_Ratio     0.049963
Area                0.072686
Other               0.077779
Climate             0.119464
Density             0.196251
Net_Migration       0.381599
Literacy            0.516169
Service             0.529802
Phones              0.830849
GDP_per_Capita      1.000000
Name: GDP_per_Capita, dtype: float64
```

Pairplot By Seaborn:

Pairplot

In [15]:
```
import seaborn as sns
#Pairplot for Visualising the Overlapping of data
sns.pairplot(data)
```

Out[15]:
```
<seaborn.axisgrid.PairGrid at 0x16858ad1dc0>
```



Data Splitting for Models Application:

Splitting Data With No Scaling:

e) Splitting Data For Applying Models

In [49]:
```
from sklearn.model_selection import train_test_split
#final dataset split without scaling
#'GDP_per_Capita' and 'Country' columns are going to be separted from other columns
y = data['GDP_per_Capita']
X = data.drop(['GDP_per_Capita','Country', 'Region'], axis=1)
#random_state acts like a numpy seed, it is used for data reproducibility.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=100)
```

In [50]:
```
#Training dataset shape = (row,columns)
X_train.shape
```

Out[50]:
```
(181, 17)
```

In [51]:
```
#Testing dataset shape = (row,columns)
X_test.shape
```

Out[51]:
```
(46, 17)
```

Splitting Data With Scaling:

In [52]:
```
from sklearn.preprocessing import StandardScaler
#Standard Scalar transforms the data in such a manner that it has mean as 0 and standard deviation as 1.
#If you want more normally distributed data [(value - mean)/std].
scaler = StandardScaler()

X2_train = scaler.fit_transform(X_train)
X2_test = scaler.fit_transform(X_test)
y2_train = y_train
y2_test = y_test
```

In [53]:
```
#Displaying Standard data
print(X2_train)
print(X2_test)
print(y2_train)
print(y2_test)
```

```
[[-0.26691867 -0.31226486 -0.19027497 ...  0.82295111  0.82773996
  -1.32732979]
 [-0.29155009 -0.33740288  0.48247815 ... -1.02444984 -1.1932454
   1.79252646]
 [ 0.16649578  1.00431806 -0.20832788 ...  1.78389757 -0.44714686
  -1.04908834]
 ...
 [-0.28220266 -0.32764849 -0.1896544  ... -0.43596325 -0.93971677
   1.11764295]
 [-0.19227372  0.25080925 -0.21301036 ... -0.14544455  0.63216073
  -0.39788494]
 [ 0.15187074  1.14411473 -0.20951261 ... -0.39126806  0.67562278
  -0.23804411]]
[[-1.82216838e-01 -9.22123297e-02 -3.14965937e-01 -4.82783482e-01
   1.82478236e-02  1.39380739e+00  7.89604131e-02 -9.06928826e-01
  -1.00271820e+00 -4.93654246e-01  9.43795745e-01  1.35172167e-02
   1.41686203e+00  4.29881591e-01 -6.50508423e-01  1.64690887e+00
  -1.07949599e+00]
 [-1.04129604e-01  8.09173327e-02 -2.95705286e-01 -4.35532333e-01
```

```
In [54]: #Displaying the number of (rows, columns) in X, X_train, X_test
         X.shape, X_train.shape, X_test.shape

Out[54]: ((227, 17), (181, 17), (46, 17))

In [55]: #Displaying the number of (rows, columns) in y, y_train, y_test
         y.shape, y_train.shape, y_test.shape

Out[55]: ((227,), (181,), (46,))
```

Applying Models:

The models we are going to train are as follows:

- Linear Regression
- Random Forest
- Gradient Boosting
- SVM

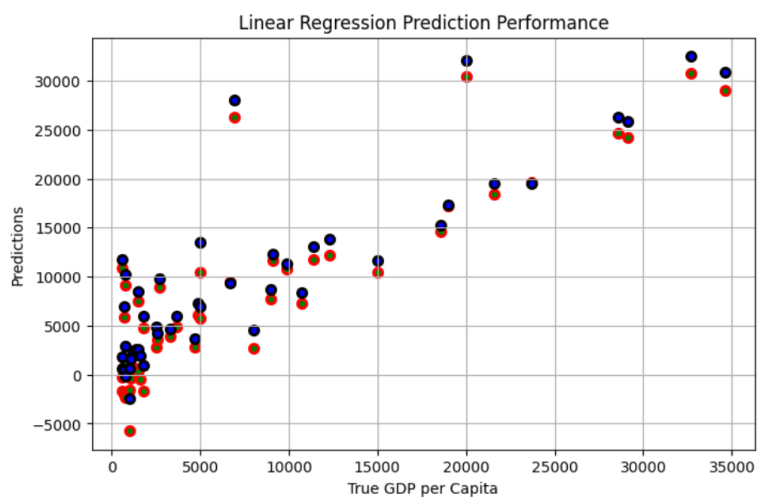
Linear Regression:

```
In [77]: #We will try linear regression, and compare the results as a reference that other models have better result or not.
         #Applying linear Regression Model.......

         lr1 = LinearRegression()
         lr1.fit(X_train,y_train)

         lr2 = LinearRegression()
         lr2.fit(X2_train,y2_train)

Out[77]: LinearRegression()
```

```
In [78]: #Prediction
         lr1_pred = lr1.predict(X_test)
         lr2_pred = lr2.predict(X2_test)
```

```
In [99]: print('Linear Regression Performance:')

         print('\nNo scaling:')
         print('MAE:', metrics.mean_absolute_error(y_test, lr1_pred))
         print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, lr1_pred)))
         print('R2_Score: ', metrics.r2_score(y_test, lr1_pred))

         print('\nWith scaling:')
         print('MAE:', metrics.mean_absolute_error(y2_test, lr2_pred))
         print('RMSE:', np.sqrt(metrics.mean_squared_error(y2_test, lr2_pred)))
         print('R2_Score: ', metrics.r2_score(y2_test, lr2_pred))

         fig = plt.figure(figsize=(8, 5))
         plt.scatter(y_test,lr1_pred,color='green', linewidths=2, edgecolors='red')
         plt.scatter(y2_test,lr2_pred,color='blue', linewidths=2, edgecolors='black')
         plt.xlabel('True GDP per Capita')
         plt.ylabel('Predictions')
         plt.title('Linear Regression Prediction Performance')
         plt.grid()
         plt.show()
```

```
Linear Regression Performance:

No scaling:
MAE: 3471.801312823844
RMSE: 4886.85570560163
R2_Score:  0.733249406655962

With scaling:
MAE: 3338.208727040831
RMSE: 5104.400113842092
R2_Score:  0.708971325090908
```



Random Forest:

2. Random Forest

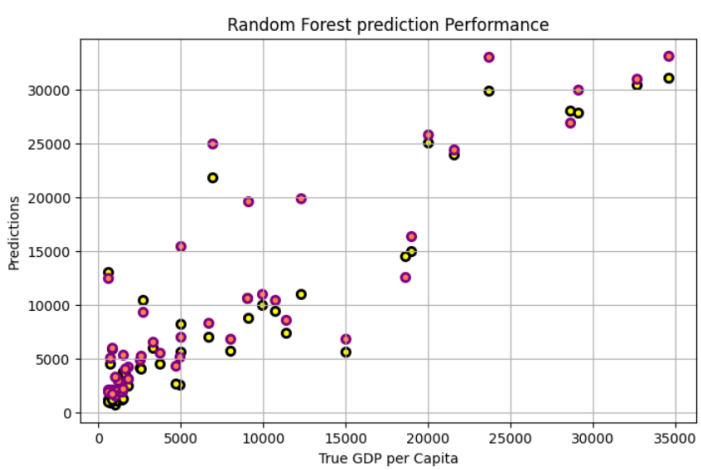
In [98]:
```
from sklearn.ensemble import RandomForestRegressor

#Let's try Random Forest with our Data Splits.
#Scaling is not going to be tested for Random Forest,since it should not affect this algorithm's performance.
#Applying Random Forest Regressor Model.......

rf1 = RandomForestRegressor(random_state=101, n_estimators=200)
rf1.fit(X_train, y_train)

rf2 = RandomForestRegressor(random_state=101, n_estimators=200)
rf2.fit(X2_train, y2_train)
```

Out[98]:
```
RandomForestRegressor(n_estimators=200, random_state=101)
```

In [99]:
```
#Predictions
rf1_pred = rf1.predict(X_test)
rf2_pred = rf2.predict(X2_test)
```

In [100]:
```
print('Random Forest Performance:')

print('\nNo scaling:')
print('MAE:', metrics.mean_absolute_error(y_test, rf1_pred))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, rf1_pred)))
print('R2_Score: ', metrics.r2_score(y_test, rf1_pred))

print('\nwith scaling:')
print('MAE:', metrics.mean_absolute_error(y2_test, rf2_pred))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y2_test, rf2_pred)))
print('R2_Score: ', metrics.r2_score(y2_test, rf2_pred))

fig = plt.figure(figsize=(8, 5))
plt.scatter(y2_test,rf1_pred,color='yellow', linewidths=2, edgecolors='black')
plt.scatter(y2_test,rf2_pred,color='coral', linewidths=2, edgecolors='purple')
plt.xlabel('True GDP per Capita')
plt.ylabel('Predictions')
plt.title('Random Forest prediction Performance')
plt.grid()
plt.show()
```

```
Random Forest Performance:

No scaling:
MAE: 2603.4239130434785
RMSE: 4074.119432066165
R2_Score:  0.8145982305091836

with scaling:
MAE: 3484.8478260869565
RMSE: 5071.4484231524675
R2_Score:  0.7127166948446086
```



Hyperparameter Tuning for Random Forest

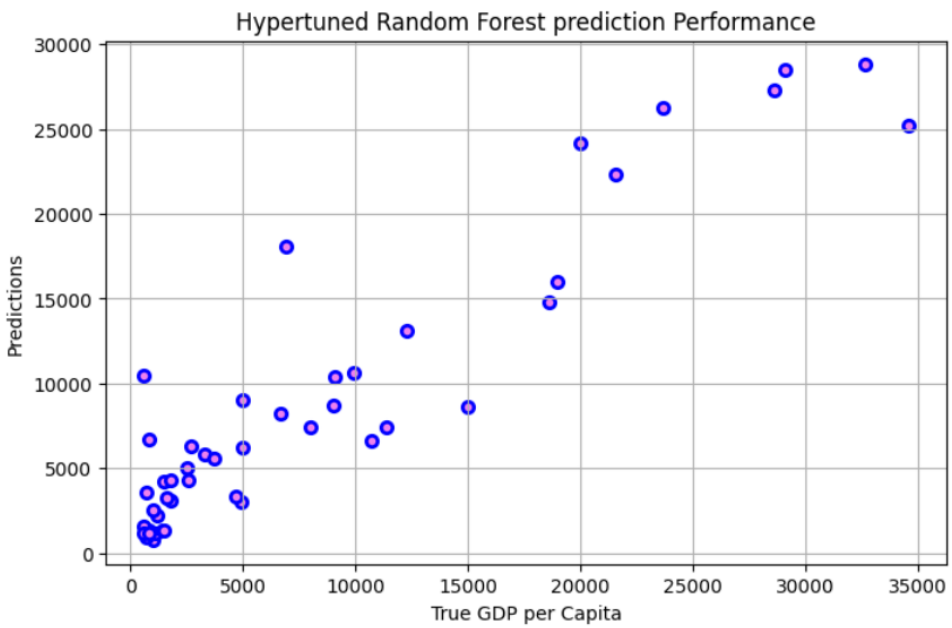
In [101]:
```
from sklearn.model_selection import GridSearchCV
rf_param_grid = {'max_features': ['sqrt', 'auto'],
              'min_samples_leaf': [1, 3, 5],
              'n_estimators': [90, 900, 1000, 1050],
             'bootstrap': [False, True]}
rf_grid = GridSearchCV(estimator= RandomForestRegressor(), param_grid = rf_param_grid,  n_jobs=-1, verbose=0)
rf_grid.fit(X_train,y_train)
#rf_grid.fit(X2_train,y2_train)

print(rf_grid.best_params_)
```

```
{'bootstrap': False, 'max_features': 'sqrt', 'min_samples_leaf': 3, 'n_estimators': 1000}
```

In [23]:
```
rf_grid_predictions = rf_grid.predict(X_test)
```

In [24]:
```
print('MAE:', metrics.mean_absolute_error(y_test, rf_grid_predictions))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, rf_grid_predictions)))
print('R2_Score: ', metrics.r2_score(y_test, rf_grid_predictions))
fig = plt.figure(figsize=(8, 5))
plt.scatter(y_test,rf_grid_predictions,color='violet', linewidths=2, edgecolors='blue')
plt.xlabel('True GDP per Capita')
plt.ylabel('Predictions')
plt.title('Hypertuned Random Forest prediction Performance')
plt.grid()
plt.show()
```

```
MAE: 2463.864734299517
RMSE: 3530.0309514958253
R2_Score:  0.8608114889081677
```



Gradient Boosting:

3. Gradient Boosting

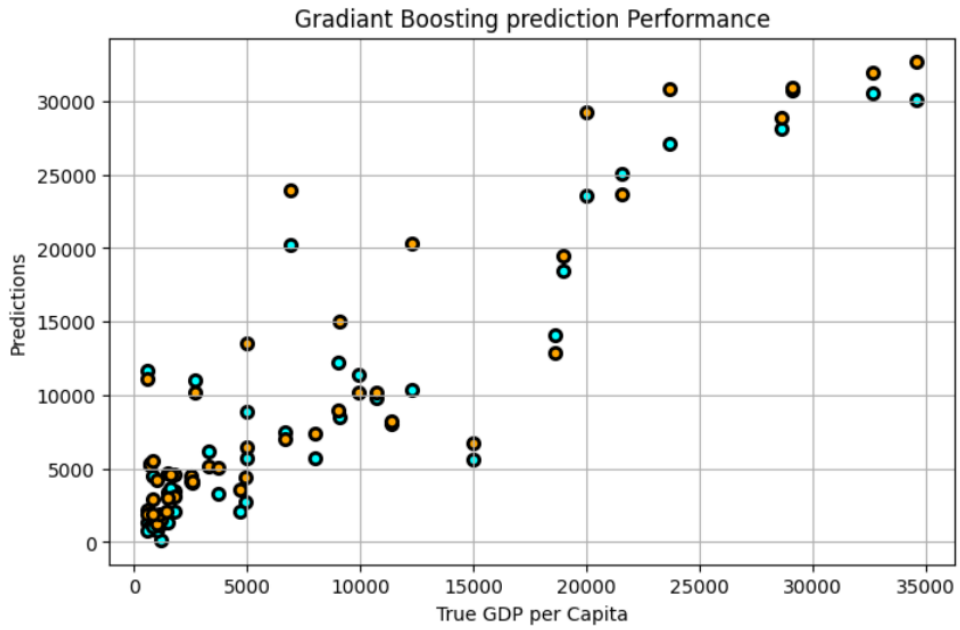
In [65]:
```
from sklearn.ensemble import GradientBoostingRegressor
#Now Let's try Gradient Boosting with our Data Splits.
#Scaling is not going to be tested for Gradient Boosting,since it should not affect this algorithm's performance.
#Applying Gradient Boosting Regressor Model.......

gb1 = GradientBoostingRegressor(learning_rate=0.1, n_estimators=100, min_samples_split=2, random_state=101)
gb1.fit(X_train, y_train)

gb2 = GradientBoostingRegressor(learning_rate=0.1, n_estimators=100, min_samples_split=2, random_state=101)
gb2.fit(X2_train, y2_train)
```

Out[65]:
```
GradientBoostingRegressor(random_state=101)
```

In [66]:
```
#predictions
gb1_pred = gb1.predict(X_test)
gb2_pred = gb2.predict(X2_test)
```

In [67]:
```
print('Gradiant Boosting Performance:')

print('\nNo scaling:')
print('MAE:', metrics.mean_absolute_error(y_test, gb1_pred))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, gb1_pred)))
print('R2_Score: ', metrics.r2_score(y_test, gb1_pred))

print('\nwith scaling:')
print('MAE:', metrics.mean_absolute_error(y2_test, gb2_pred))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y2_test, gb2_pred)))
print('R2_Score: ', metrics.r2_score(y2_test, gb2_pred))

fig = plt.figure(figsize=(8, 5))
plt.scatter(y2_test,gb1_pred,color='cyan', linewidths=2, edgecolors='k')
plt.scatter(y2_test,gb2_pred,color='orange', linewidths=2, edgecolors='black')
plt.xlabel('True GDP per Capita')
plt.ylabel('Predictions')
plt.title('Gradiant Boosting prediction Performance')
plt.grid()
plt.show()
```

```
Gradiant Boosting Performance:

No scaling:
MAE: 2500.869527593971
RMSE: 3808.238370382432
R2_Score:  0.8380076105166444

with scaling:
MAE: 3122.5895834772678
RMSE: 4667.823786003698
R2_Score:  0.7566253903292285
```



Hyperparameter Tuning for Gradient Boosting

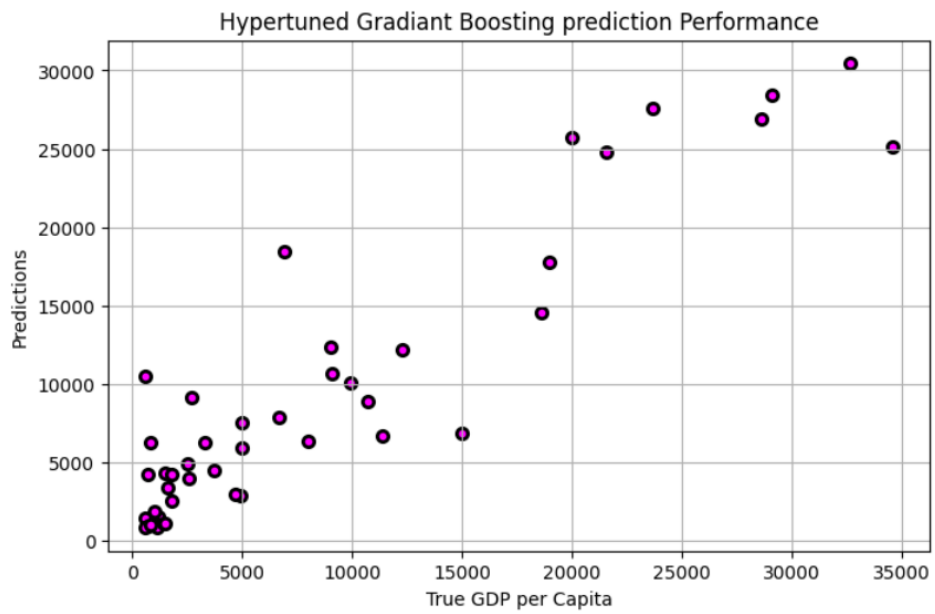
In [111]:
```
#We will use grid search in order to obtain good parameters for our GBM regressor. Of course our optimization
gb_param_grid = {'learning_rate':[1,0.1, 0.01, 0.001],
                 'n_estimators':[400, 500, 600],
                 'max_depth':[4, 5, 7],
                 'subsample':[0.7, 0.8],
                 'min_samples_leaf':[1, 20],
                 'min_samples_split':[10, 20],
                 'max_features':[5, 7]}

gb_grid = GridSearchCV(estimator =GradientBoostingRegressor(random_state=101), param_grid = gb_param_grid,
                       n_jobs=-1, cv=5)

gb_grid.fit(X_train,y_train)
print(gb_grid.best_params_)
```

```
{'learning_rate': 0.01, 'max_depth': 5, 'max_features': 5, 'min_samples_leaf': 1, 'min_samples_split': 10, 'n_estimators': 600,
'subsample': 0.8}
```

In [112]:
```
gb_grid_predictions = gb_grid.predict(X_test)
```

In [113]:
```
print('MAE:', metrics.mean_absolute_error(y_test, gb_grid_predictions))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, gb_grid_predictions)))
print('R2_Score: ', metrics.r2_score(y_test, gb_grid_predictions))
fig = plt.figure(figsize=(8, 5))
plt.scatter(y_test,gb_grid_predictions,color='magenta', linewidths=2, edgecolors='black')
plt.xlabel('True GDP per Capita')
plt.ylabel('Predictions')
plt.title('Hypertuned Gradiant Boosting prediction Performance')
plt.grid()
plt.show()
```

```
MAE: 2579.2078749642505
RMSE: 3763.2444504669106
R2_Score:  0.8418128424689382
```



SVM(Support Vector Machine):

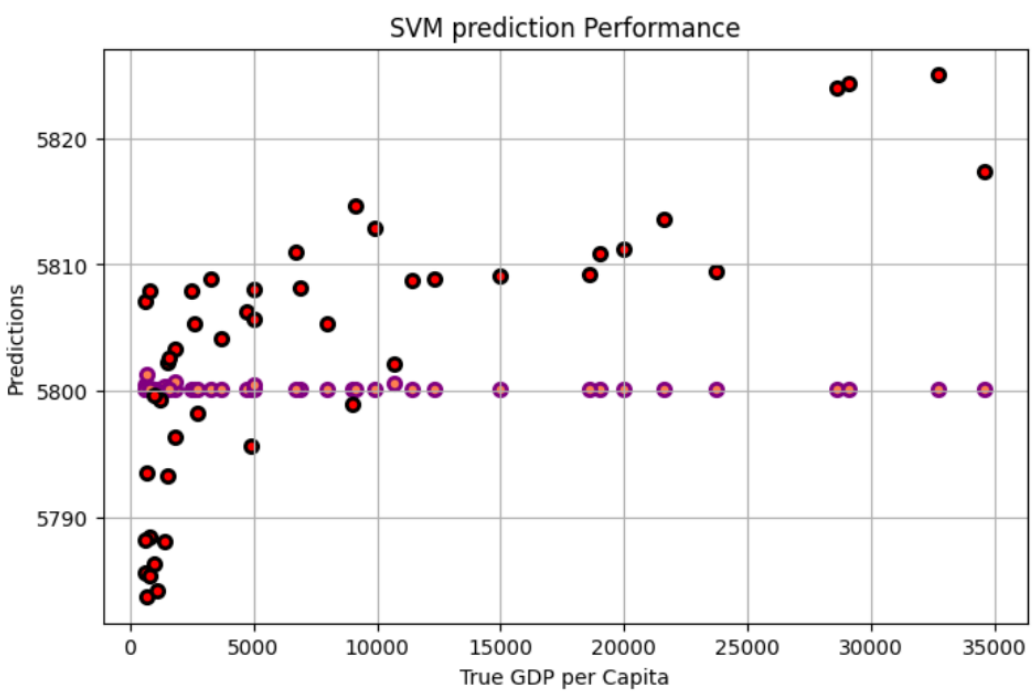
We will use the Regressor Function of SVM….

```
In [32]: #We will try SVM Regressor, and compare the results as a reference that other models have better result or not.
         #Applying SVM Regressor Model.......

         svm1 = SVR(kernel='rbf')
         svm1.fit(X_train,y_train)

         svm2 = SVR(kernel='rbf')
         svm2.fit(X2_train,y2_train)

Out[32]: SVR()
```

```
In [94]: #Predictions
         svm1_pred = svm1.predict(X_test)
         svm2_pred = svm2.predict(X2_test)
```

In [100]:
```
print('SVM Performance:')
print('\nNo scaling:')
print('MAE:', metrics.mean_absolute_error(y_test, svm1_pred))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, svm1_pred)))
print('R2_Score: ', metrics.r2_score(y_test, svm1_pred))

print('\nWith scaling:')
print('MAE:', metrics.mean_absolute_error(y2_test, svm2_pred))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y2_test, svm2_pred)))
print('R2_Score: ', metrics.r2_score(y2_test, svm2_pred))

fig = plt.figure(figsize=(8, 5))
plt.scatter(y_test,svm1_pred,color='coral', linewidths=2, edgecolors='purple')
plt.scatter(y2_test,svm2_pred,color='red', linewidths=2, edgecolors='black')
plt.xlabel('True GDP per Capita')
plt.ylabel('Predictions')
plt.title('SVM prediction Performance')
plt.grid()
plt.show()
```

```
SVM Performance:

No scaling:
MAE: 6943.559244462341
RMSE: 9781.030968856261
R2_Score:  -0.06860113346189523

With scaling:
MAE: 6937.196128156793
RMSE: 9772.493599878293
R2_Score:  -0.06673649144740446
```



Conclusion:

In this project, we used countries_of_the_world dataset to build a GDP Predictor. 4 different learning regressors (Linear Regression, SVM, Random Forest, and Gradient Boosting) were tested, and we have achieved the best prediction performance using Random Forest, followed by Gradient Boosting, and then Linear Regression, while SVM achieved the worst performance of the 4.

The best prediction performance was achieved using **Random Forest Regressor,** using all features in the dataset, and resulted in the following metrics:

- Mean Absolute Error __(MAE)__: 2142.13
- Root mean squared error __(RMSE)__: 3097.19
- R2-squared Score __(R2_Score)__: 0.8839

Taking into account that the gdp_per_capita values in the dataset ranges from 500 to 55100 USD.

Input From User:

In [68]:
```
import numpy as np
input_data = (71201, 468, 152.1, 0.00, 6.60, 4.50, 100.000000, 497.2, 2.22, 0, 97.78, 3, 8.71, 6.25, 0.170, 0.030, 0.800)

input_data_as_numpy = np.asarray(input_data)

input_data_reshaped = input_data_as_numpy.reshape(1, -1)

prediction = rf1.predict(input_data_reshaped)

print(prediction)
```

```
[22730.]
```

```
D:\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have valid feature names, but RandomForestRegressor
was fitted with feature names
  warnings.warn(
```

In [69]:
```
import pickle

filename = 'trained GDP_Predictor.sav'
pickle.dump(rf1, open(filename, 'wb'))
```

In [70]:
```
trained_model = pickle.load(open('trained GDP_Predictor.sav', 'rb'))
```

In [72]:
```
input_data = (71201, 468, 152.1, 0.00, 6.60, 4.50, 100.000000, 497.2, 2.22, 0, 97.78, 3, 8.71,
              6.25, 0.170, 0.030, 0.800)

input_data_as_numpy = np.asarray(input_data)

input_data_reshaped = input_data_as_numpy.reshape(1, -1)

prediction = trained_model.predict(input_data_reshaped)

print('GDP_per_Capita is as:')
print(prediction)
```

```
GDP_per_Capita is as:
[22730.]
```

```
D:\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have valid feature names, but RandomForestRegressor
was fitted with feature names
  warnings.warn(
```